SENAI Serviço Nacional de Aprendizagem Industrial

Atividade Avaliativa N1:
Um Problema, Dois Paradigmas

Disciplina: Paradigmas de Programação
Turma: Engenharia de Software - Turma 6
Integrantes: Gabriel Machado Chiles e Pabline Pereira dos Santos

Goiânia - GO,
Agosto de 2026

1. Sumário

2. Resumo do Projeto **2**
3. Problema de Mercado **2**
4. Escopo e Requisitos **2**
Requisitos Funcionais (RF): 2
Regras de Negócio (RN): 2
5. Paradigma 1 + Linguagem 1 **3**
6. Paradigma 2 + Linguagem 2 **3**
7. Modelagem das Duas Soluções **3**
Abordagem Orientada a Objetos (C#): 3
Abordagem Imperativa (C): 4
8. Comparação Técnica **4**
9. Plano de Implementação Futura **5**
10. Conclusão **6**
11. Referências **6**

2. Resumo do Projeto

O presente projeto tem como objetivo o planejamento de um sistema denominado “Gerenciador de Chamados Básico”, voltado para pequenas equipes de suporte técnico de Tecnologia da Informação. O público-alvo da solução compreende dois perfis de usuário: os colaboradores da empresa, que são os solicitantes dos chamados, e os técnicos de TI, responsáveis por atender e solucionar os problemas relatados.

A solução proposta consiste em uma aplicação de interface textual (terminal), por meio da qual é possível registrar solicitações de suporte, atribuir um técnico responsável a cada chamado, acompanhar e alterar o status do atendimento, além de armazenar o histórico da solução técnica aplicada. Dessa forma, o sistema busca eliminar a desorganização observada atualmente no fluxo de solicitações internas de suporte, promovendo maior controle, rastreabilidade e transparência ao longo de todo o processo de atendimento.

3. Problema de Mercado

Em muitas pequenas e médias empresas, o suporte técnico interno ainda é conduzido de maneira informal, por meio de canais como conversas presenciais, ligações telefônicas ou aplicativos de mensagens instantâneas. Essa prática, embora pareça prática no curto prazo, traz diversas consequências negativas para a organização: chamados são esquecidos ou perdidos, não há critério claro de priorização entre as demandas, e inexiste um histórico documentado das soluções já aplicadas a problemas recorrentes.

Como consequência direta desse cenário, a equipe de TI frequentemente retrabalha soluções já conhecidas, os solicitantes não têm visibilidade sobre o andamento de suas requisições, e a gestão da equipe de suporte carece de indicadores mínimos de acompanhamento e desempenho. Diante disso, identifica-se a necessidade de um sistema simples, de baixo custo de implementação e fácil manutenção, capaz de centralizar o registro dos chamados, padronizar o fluxo de atendimento e preservar o conhecimento técnico gerado a cada solução — ainda que, nesta primeira etapa, a proposta esteja limitada a um ambiente de terminal, sem recursos gráficos ou de rede.

4. Escopo e Requisitos

O escopo do projeto abrange exclusivamente as funcionalidades essenciais de registro e acompanhamento de chamados técnicos, operando em ambiente de

terminal/console. Ficam fora do escopo desta etapa: (i) mecanismos de autenticação e controle de senha de usuários; (ii) controle de inventário de peças, equipamentos ou hardware; e (iii) o envio automático de notificações, como e-mails ou mensagens instantâneas, aos envolvidos no processo. Tais limitações são intencionais e visam manter o projeto compatível com o porte proposto pela atividade avaliativa, sem comprometer a análise comparativa entre paradigmas, que constitui o real objetivo desta N1

Requisitos Funcionais (RF):

- **RF1 — Registrar Chamado:** Cadastrar um novo chamado contendo o nome do solicitante e a descrição do problema, **registrando automaticamente a data/hora de abertura** (para garantir a rastreabilidade prometida).
- **RF2 — Listagem e Visibilidade:** Listar os chamados aplicando filtros básicos (por status, por solicitante ou por técnico). *Justificativa: Atende à necessidade de dar visibilidade aos solicitantes sobre suas requisições e organizar a fila.*
- **RF3 — Atribuição de Responsável:** Atribuir ou alterar o técnico responsável por um chamado específico.
- **RF4 — Atualização de Status:** Alterar o status atual de um chamado (Aberto, Em Andamento, Resolvido), **registrando a data/hora da alteração** para compor o histórico do atendimento.
- **RF5 — Resolução e Base de Conhecimento:** Finalizar um chamado, registrando obrigatoriamente a solução técnica adotada e a data de encerramento.
- **RF6 — Consulta de Histórico (Evitar Retrabalho):** Permitir a busca ou listagem exclusiva de chamados "Resolvidos", exibindo o problema e a solução técnica aplicada. *Justificativa: Atende diretamente à dor de "retrabalhar soluções já conhecidas" e "preservar o conhecimento técnico".*
- **RF7 — Indicadores Básicos:** Exibir um painel de resumo gerencial em texto mostrando a quantidade de chamados totais por status (ex: X Abertos, Y Em Andamento, Z Resolvidos) e chamados resolvidos por técnico. *Justificativa: Atende à necessidade de "indicadores mínimos de acompanhamento e desempenho" citada no texto.*

Regras de Negócio (RN):

- **RN1 — Restrição de Status:** Os status do chamado são estritamente restritos: “Aberto”, “Em Andamento” e “Resolvido”.
- **RN2 — Fluxo de Atendimento:** Um chamado só pode ter seu status alterado para "Em Andamento" se já houver um técnico responsável atribuído a ele.
- **RN3 — Obrigatoriedade de Documentação:** O sistema bloqueará a alteração do status

para “Resolvido” caso o campo referente à solução técnica adotada esteja vazio.

- **RN4 — Limite de Trabalho em Progresso (WIP):** Um mesmo técnico não pode assumir, simultaneamente, mais de 3 (três) chamados com status “Em Andamento”. *Justificativa: Garante que a equipe não fique sobrecarregada, mantendo o controle prometido na justificativa.*
- **RN5 — Imutabilidade de Histórico:** Uma vez que o chamado for marcado como "Resolvido", ele não poderá ser modificado ou reaberto, servindo a partir desse momento apenas como histórico e base de conhecimento. *Justificativa: Como é um sistema simples de terminal, isso simplifica a lógica de programação da N1 e garante a integridade dos dados resolvidos.*

5. Paradigma 1 + Linguagem 1

O paradigma Orientado a Objetos (POO) organiza a solução de um problema computacional a partir da identificação de entidades do mundo real, representadas por objetos que reúnem, em uma mesma estrutura, os dados (atributos) e os comportamentos (métodos) a eles relacionados. Esse modelo favorece o encapsulamento, isto é, a proteção do estado interno de um objeto, que passa a ser alterado apenas por meio de operações previamente definidas e controladas pela própria entidade.

Para representar o núcleo funcional do Gerenciador de Chamados nesse paradigma, optou-se pela linguagem C#, desenvolvida pela Microsoft e amplamente utilizada no mercado corporativo. Essa escolha justifica-se por três motivos principais: em primeiro lugar, a linguagem foi concebida desde sua origem em torno dos conceitos de orientação a objetos, o que dispensa o uso de artifícios ou adaptações; em segundo lugar, possui tipagem estática forte, reduzindo a incidência de erros em tempo de execução; e, por fim, é uma linguagem amplamente documentada, o que favorece o aprendizado e a consulta técnica por parte do grupo.

Na organização proposta, os principais elementos do sistema — o chamado, seu status, o técnico responsável e a solução aplicada — são representados por uma classe Chamado. O estado interno dessa classe (status, técnico e solução) é mantido como privado, sendo acessível apenas por meio de métodos públicos, como AtribuirTecnico() e FinalizarChamado(). Essa arquitetura garante que as regras de negócio RN2 e RN3 sejam verificadas obrigatoriamente antes de qualquer alteração de estado, impedindo que o sistema atinja uma condição inconsistente.

6. Paradigma 2 + Linguagem 2

Paradigma Imperativo | Linguagem: C

O paradigma Imperativo, por sua vez, descreve a solução de um problema como uma sequência ordenada de instruções que alteram, passo a passo, o estado da memória do computador. Diferentemente da orientação a objetos, esse paradigma não agrupa dados e comportamento em uma mesma unidade: as estruturas de dados permanecem passivas, enquanto funções externas são responsáveis por manipulá-las diretamente.

A linguagem escolhida para representar essa abordagem foi a linguagem C, criada por Dennis Ritchie e amplamente reconhecida como a representante clássica do paradigma imperativo e procedural. Sua escolha justifica-se pelo fato de a linguagem trabalhar diretamente com memória e ponteiros, sem qualquer camada de abstração orientada a objetos, o que evidencia de forma nítida as diferenças estruturais entre os dois paradigmas propostos neste trabalho.

Nessa abordagem, os dados do chamado são organizados em uma struct Chamado, contendo os mesmos campos definidos na versão orientada a objetos. Como a linguagem C não permite associar métodos diretamente a uma struct, o comportamento do sistema é implementado por meio de funções autônomas, como atribuir_tecnico() e finalizar_chamado(), que recebem um ponteiro para a struct, verificam as regras de negócio e alteram os valores diretamente na memória. Nesse modelo, o estado do chamado é exposto e mutável, dependendo inteiramente da disciplina do programador para que as regras de negócio sejam sempre respeitadas.

7. Modelagem das Duas Soluções

A seguir, apresenta-se uma representação inicial, em pseudocódigo próximo às linguagens escolhidas, da estrutura de dados e das principais operações de cada abordagem. Ressalta-se que, conforme orientação da atividade, não é exigida nesta etapa a implementação funcional completa das soluções, mas apenas a modelagem que evidencie como o mesmo núcleo de requisitos seria organizado em cada paradigma.

Abordagem Orientada a Objetos (C#):

```
class Chamado {
```

```
    private int id;
    private string descricao;
    private string status;
    private string tecnico;
    private string solucao;

    public bool AtribuirTecnico(string nomeTecnico) { ... }
    public bool FinalizarChamado(string textoSolucao) { ... }
}
```

Abordagem Imperativa (C):

```
typedef struct {
    int id;
    char descricao[100];
    char status[20];
    char tecnico[50];
    char solucao[200];
} Chamado;

int atribuir_tecnico(Chamado *c, char *nome_tecnico) { ... }
int finalizar_chamado(Chamado *c, char *texto_solucao) { ... }
```

8. Comparação Técnica

Critério	Orientado a Objetos (C#)	Imperativo (C)
Como o problema é dividido?	Em classes/objetos que modelam entidades da situação real e se comunicam entre si.	Em um fluxo de execução linear (sequência de comandos) que manipula structs.
Qual é o principal elemento da solução?	O objeto — a classe Chamado, que reúne dados e métodos em uma única unidade.	A estrutura de dados — a struct Chamado, manipulada por funções externas.
Como os dados/estado são tratados?	Estado encapsulado (protegido) e modificado apenas por	Estado exposto e mutável, modificado diretamente pelas

Critério	Orientado a Objetos (C#)	Imperativo (C)
	métodos autorizados da própria classe.	funções que acessam a memória.
Como as regras são representadas?	Como validações internas aos métodos da classe (ex.: verificação de RN2 e RN3 dentro de FinalizarChamado()).	Como verificações condicionais dentro de cada função (ex.: checagem de RN2 e RN3 em finalizar_chamado()).
Legibilidade esperada	Alta, pois o código tende a refletir diretamente as entidades e regras do domínio do problema.	Moderada, pois a leitura exige acompanhar o fluxo sequencial de chamadas entre funções e struct.
Facilidade de escrita/desenvolvimento	Facilitada pelo reaproveitamento da estrutura de classes e pelos recursos nativos de POO da linguagem.	Direta, porém exige atenção redobrada na passagem correta de parâmetros e ponteiros.
Confiabilidade e tratamento de erros	Alta: as regras são intrínsecas ao objeto, o que reduz a chance de alterações indevidas de estado.	Moderada a baixa: como o dado é exposto, um erro de implementação pode alterar o status sem passar pela validação.
Facilidade de manutenção/evolução	Alta: novas regras podem ser incorporadas dentro da própria classe, sem impacto no código externo.	Mais trabalhosa: qualquer nova regra exige revisão de todas as funções que manipulam a struct.
Adequação ao problema escolhido	Bastante adequado, dado o número de regras de negócio que se beneficiam do encapsulamento de estado.	Adequado ao escopo pequeno proposto, mas exige maior disciplina da equipe de desenvolvimento.
Principal vantagem	Reutilização e segurança dos dados por meio do encapsulamento das regras de negócio.	Controle explícito e detalhado do fluxo de execução, com processamento direto e eficiente.
Principal limitação	Maior curva de aprendizado inicial e verbosidade da linguagem em relação à abordagem imperativa.	Ausência de proteção nativa dos dados, dependendo inteiramente da disciplina da equipe de desenvolvimento.

9. Plano de Implementação Futura

- **Fase Estrutural:** Construção da Classe em C# e da Struct em C, declarando os mesmos atributos principais definidos no escopo.
- **Fase de Interface:** Criar um menu de terminal unificado (visual) com as opções de 1 a 5 para realizar as operações (RFs) simulando um banco de dados usando listas/arrays em memória.
- **Fase de Lógica:** Implementar os métodos e funções que processam as Regras de Negócio, focando no bloqueio da resolução sem texto (RN2) e no limite por técnico (RN3).
- **Validação de Núcleo Comum:** O grupo testará as duas aplicações simulando o mesmo fluxo do mundo real para comprovar que os comportamentos são idênticos em ambos os programas.

10. Conclusão

Retomando a pergunta norteadora desta atividade — “Se mudarmos o paradigma de programação, ainda conseguimos planejar uma solução adequada para o mesmo problema de mercado?” —, a resposta obtida a partir do desenvolvimento deste projeto é afirmativa.

Ao planejar a mesma solução sob os paradigmas Orientado a Objetos (C#) e Imperativo (C), observa-se claramente que o núcleo funcional do sistema — cadastrar, atribuir e finalizar chamados — permanece equivalente nas duas abordagens. A principal distinção reside na arquitetura de responsabilidades adotada por cada paradigma: enquanto o paradigma orientado a objetos concentra-se na proteção dos dados por meio do encapsulamento, atribuindo à própria entidade Chamado a responsabilidade de garantir suas regras internas, o paradigma imperativo exige que o programador controle explicitamente o estado exposto, distribuindo a solução entre estruturas de dados passivas e funções sequenciais ativas.

Conclui-se, portanto, que ambos os paradigmas são capazes de atender integralmente aos requisitos funcionais e às regras de negócio definidos para o problema escolhido, ainda que exijam da equipe de engenharia de software formas de pensar e organizar o código substancialmente distintas — o que reforça a importância de compreender bem as características de cada paradigma antes de escolhê-lo para um projeto real.

11. Referências

FATESG, SENAI. Introdução - Conceitos de Paradigma pt2. Material didático disponibilizado para a disciplina de Paradigmas de Programação.